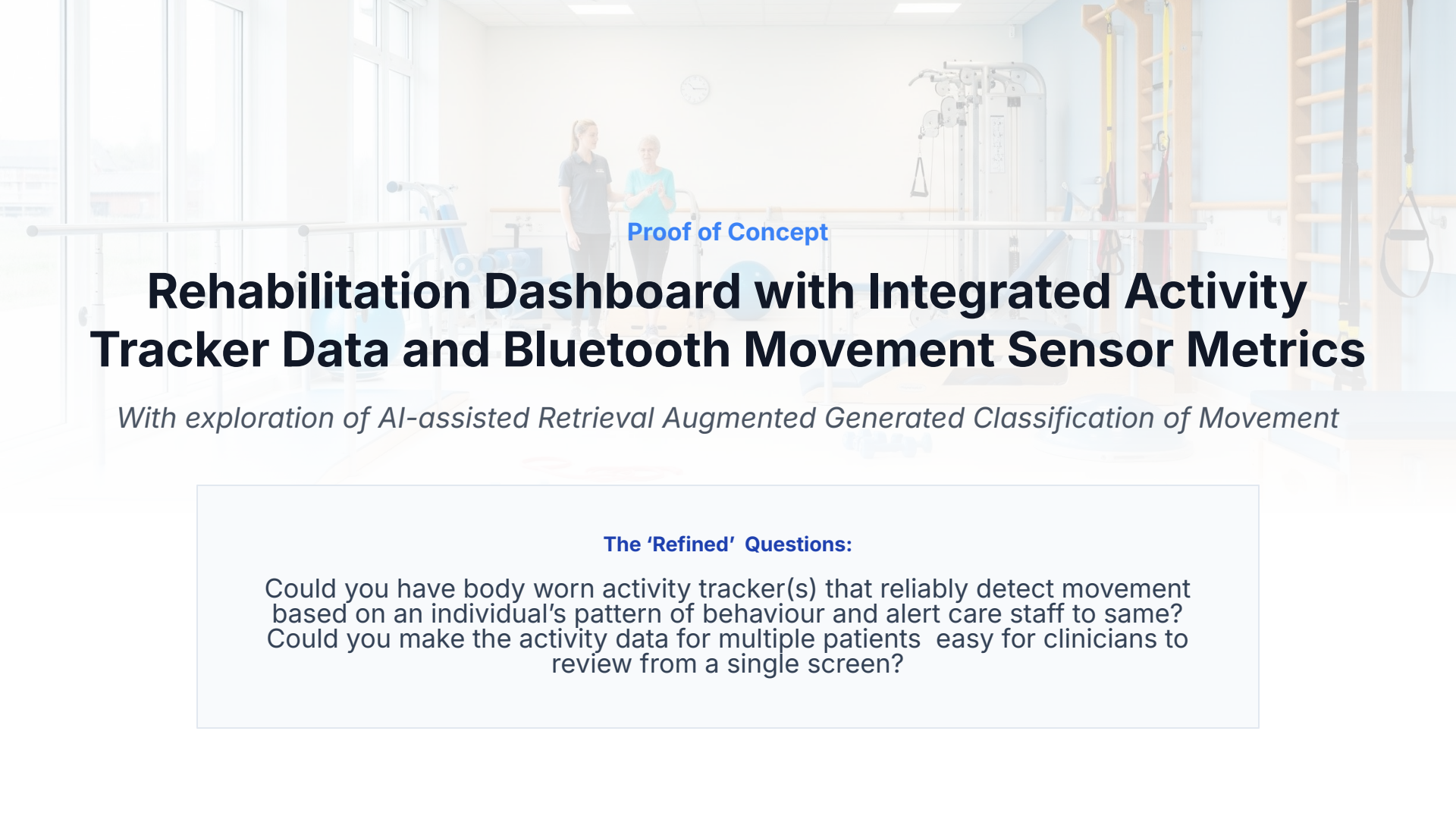




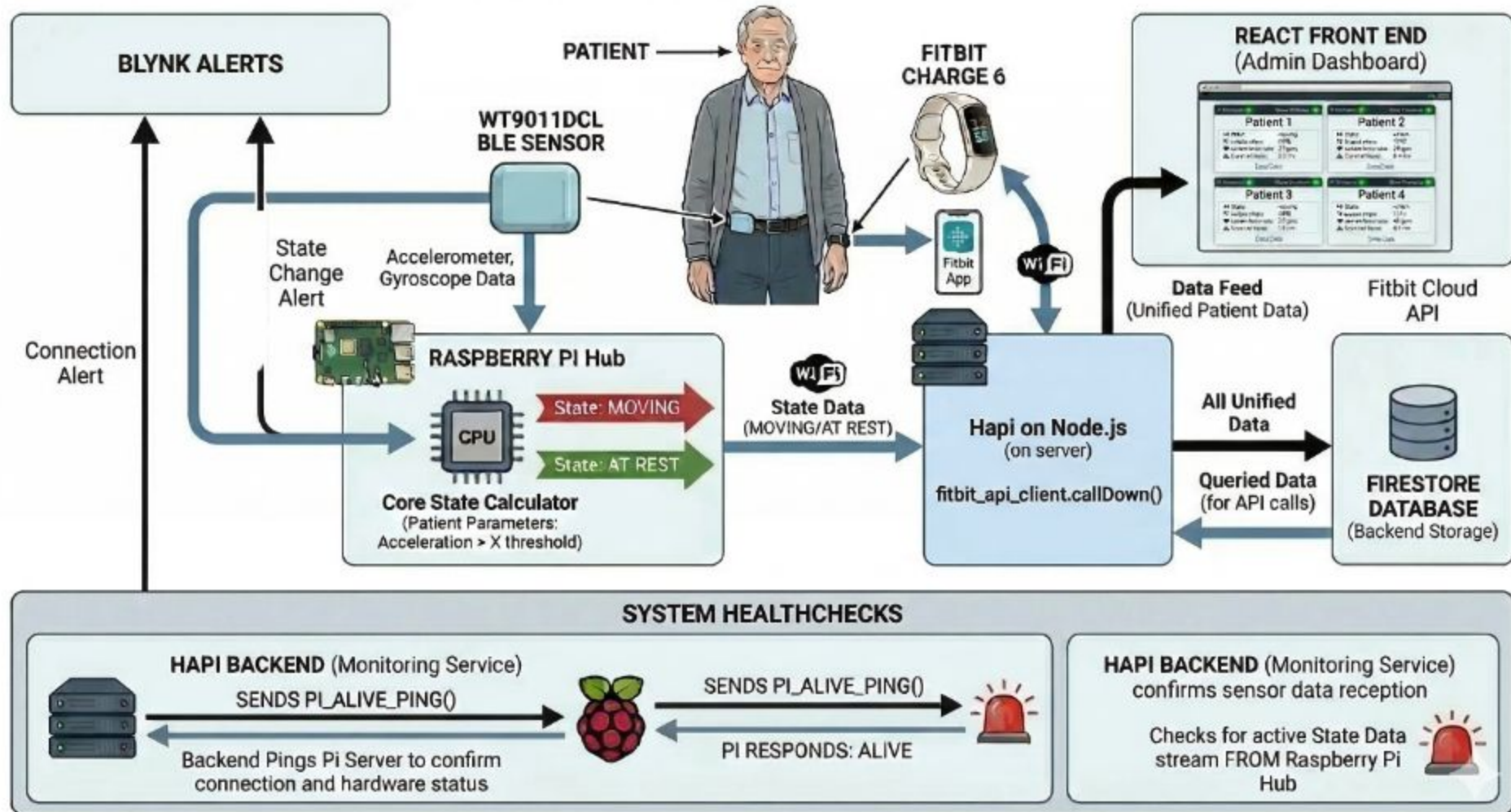
Proof of Concept

Rehabilitation Dashboard with Integrated Activity Tracker Data and Bluetooth Movement Sensor Metrics

With exploration of AI-assisted Retrieval Augmented Generated Classification of Movement

The 'Refined' Questions:

Could you have body worn activity tracker(s) that reliably detect movement based on an individual's pattern of behaviour and alert care staff to same?
Could you make the activity data for multiple patients easy for clinicians to review from a single screen?



1. Architecture Summary

SYSTEM PIPELINE

Sensor → Raspberry Pi+ Fitbit API → Backend (Node/Hapi) → Firestore → React Frontend



Architecture Model

Hybrid ingestion model (push + pull)
Backend unifies patient data streams
Event-based storage (state changes)



Layer Separation

Strict separation between acquisition, processing, and presentation layers for modular scalability.



Storage Strategy

Edge processing reduces noise and backend load.
Event-based storage records state changes rather than raw streams.

2. Frontend Architecture

CORE DATA FLOW

Service → Page (state) → Component (props) → UI



Routing & Entry

- App.tsx: Routing entry point via react-router-dom
- Minimal App.tsx focusing solely on routing
- URL paths map directly to page components



Views & Components

- Pages handle full-screen data orchestration
- Reusable UI elements (PatientCard, AppLayout)
- Separation between routing and presentation



Services & Types

- Services (patientService.ts) manage API calls
- Types define contracts for backend data
- Clear separation of data fetching logic

Summary: Robust modular design ensuring high maintainability and a clear separation of concerns across the frontend stack.

3. Backend Architecture

TECHNOLOGY STACK

Node.js (Hapi) | Firestore (NoSQL) | API-First Design



Structure & SOC

Routes: Define API endpoints (/patients, /telemetry)

Controllers: Process requests and coordinate logic

Services: Service-based structure (partially implemented, with some logic still in handlers in this PoC)



Firestore (NoSQL)

Stores records and telemetry events

Supports flexible schemas for clinical data

Enables real-time and historical access



Design Decisions

Event-based ingestion (state changes)

Lightweight endpoints for Pi communication

Decoupling edge processing from storage

Central Integration Layer: Connects Edge Devices (Pi Sensors), Cloud Storage (Firestore), and Frontend (React UI).

3. Core Strengths



End-to-End System Integration

Seamless integration from hardware sensors to user interface.



Clinical Relevance

Focused use case on rehabilitation monitoring for real-world impact.



Effective Edge Design

Optimized computing at the edge for low latency and privacy.



Separation of Concerns

Clean modular architecture for high maintainability and scalability.

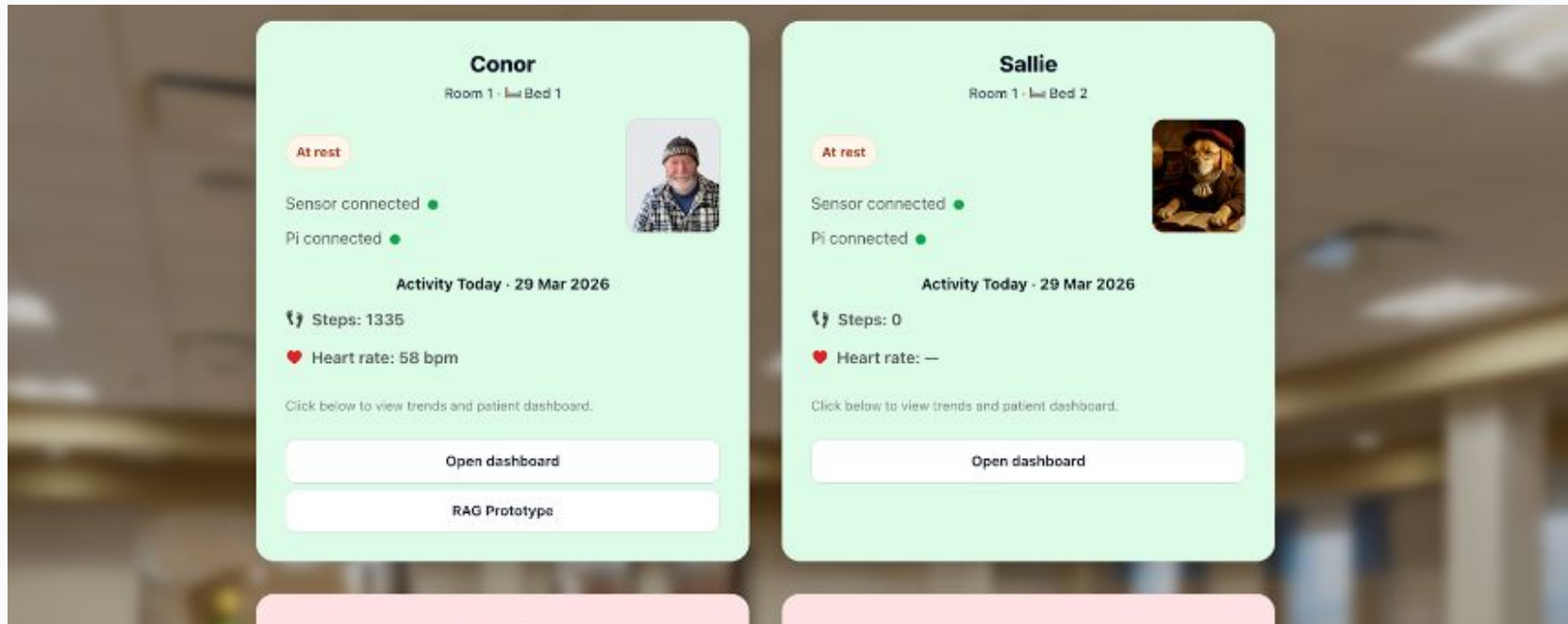


Robust Alert Logic

Attempted alert system with hysteresis to minimize false positives and ensure reliability.

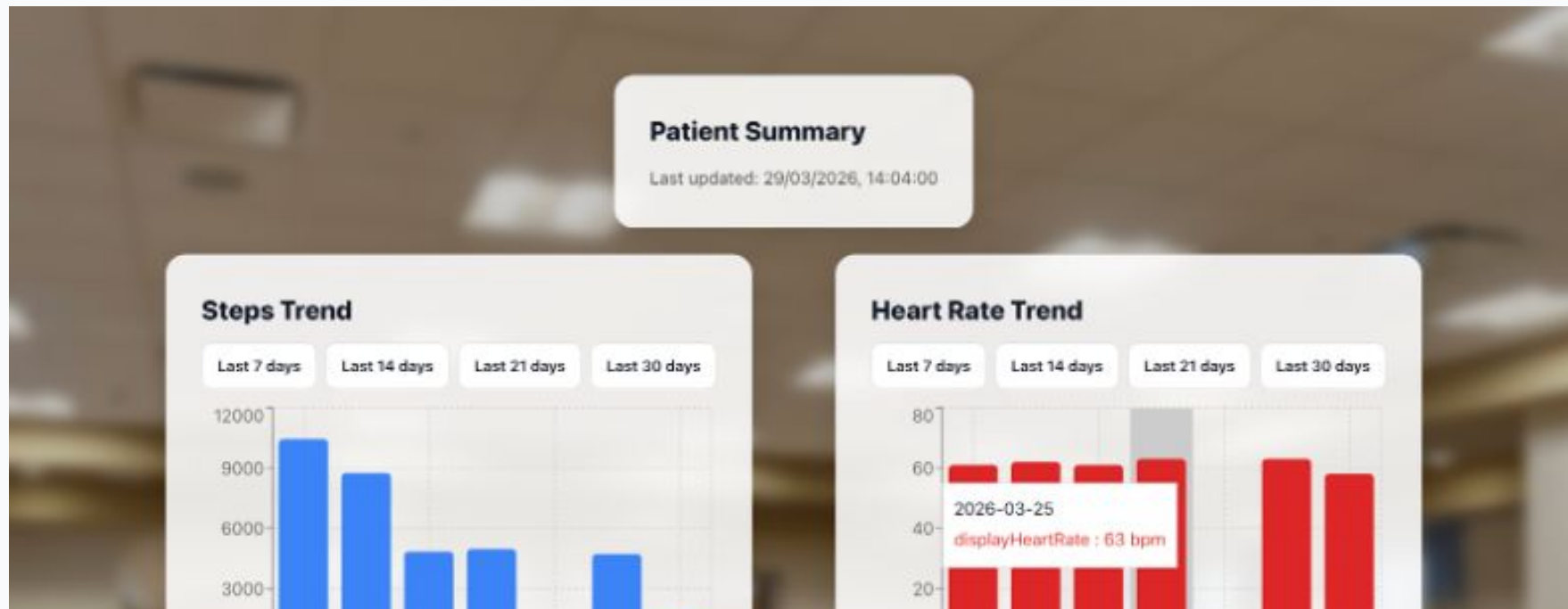
Key Differentiators: Delivering a robust, clinically-relevant telemetry solution through expert engineering and design.

Patient List View



UI Overview: Centralized monitoring interface for real-time patient health and activity status.

Patient Dashboard



Detailed Telemetry View: Real-time monitoring of activity trends and vital signs for individual patients.

4. Weaknesses and Limitations

Enhanced Sensor Data



Opportunity to combine gyro readings with directional accelerometer data. RAG-based classification is exploratory and not yet integrated into the live pipeline

Security & Inclusivity



No authentication or role-based access control implemented (PoC limitation).

Backend Refinement



Some duplication in backend logic and Pi layer configuration is hardcoded.

Testing & Data Schema



Lack of automated testing and the Firestore schema requires further refinement.

Roadmap Focus: Addressing these technical and design gaps is a priority for future iterations of the PoC.